Ride-Hailing Order and Trip Dispatch Database

(Topic 5)

CDS 534 Database Management — Group Project

Group: **Royal Family**

A · Business & Requirements	CHEN Yinan (3162550)
B · ER Modelling	ZHANG Yijie (3162641)
C · Implementation	CHEN Yanwu (3162744)
D · Queries & Experiments	LING Tong (3162677)
E · PM & Integration	HU Zhongyuan (3162304)

November 11, 2026

Contents

1 Business Context and Goals **2**
- 1.1 Business Context . . . 2
- 1.2 Stakeholder Requirements . . . 2
- 1.3 Goals and Constraints . . . 2
- 1.4 Dataset Selection . . . 3

2 Entity-Relationship Model **3**
- 2.1 ER Diagram and Notation . . . 3
- 2.2 The Five Base Tables . . . 3
- 2.3 The Three Extension Tables . . . 4
- 2.4 Normalization . . . 4

3 Implementation **4**
- 3.1 Environment and Architecture . . . 4
- 3.2 DDL Organisation and Key Constraints . . . 5
- 3.3 Data Generation and Import . . . 5
- 3.4 Verification . . . 5

4 SQL Queries and Conceptual Q&A **6**
- 4.1 Business Queries . . . 6
- 4.2 Conceptual Q&A . . . 7

5 Evaluation **9**
- 5.1 Test Environment . . . 9
- 5.2 Experiment 1: Index Before/After . . . 9
- 5.3 Experiment 2: JOIN vs Subquery . . . 9
- 5.4 Experiment 3: Concurrent Transactions . . . 10
- 5.5 Descriptive Distribution Check against RideAustin . . . 10
- 5.6 Results Summary and Goal Recheck . . . 11

6 Conclusion and Limitations **11**
- 6.1 Goal Recheck . . . 11
- 6.2 Limitations . . . 12
- 6.3 Future Work . . . 12

A Contribution Statement **13**

B Full Build Script and Query Listings **13**
- B.1 Query Listings . . . 17
- B.2 Experiment Statements . . . 19

C Full-Page ER Diagram **20**

D Execution and Result Screenshots **20**

1 Business Context and Goals

1.1 Business Context

This project builds a relational database for a ride-hailing platform to support trip queries, payment reconciliation and operational reporting. It retains five base entities, `Passenger`, `Driver`, `Vehicle`, `Trip` and `Payment`, and adds `trip_detail`, `service_tier` and `pickup_zone`. The extension tables associate service tiers, pickup zones and optional ratings with trips, giving eight entities in total.

Foreign key and unique constraints ensure valid relationships between records. Each driver is assigned one vehicle, each vehicle has at most one driver, and each trip has at most one payment. The generator also creates one detail record for every trip.

The recorded workflow begins after successful dispatch, when the driver accepts the request. Unmatched requests therefore fall outside the database scope. Trip status progresses through `accepted`, `en_route` and `completed`. Early termination after boarding is classified as completed. Before boarding, cancellation may be free or charged. The course-defined cancellation fee is USD 1.50, subject to a waiver for excessive pickup delays. Actual charges are recorded in `Payment` [5, 4]. Figure 15 in Appendix D summarises the workflow.

1.2 Stakeholder Requirements

Passengers need trip histories and fare/payment reconciliation. Drivers need cumulative trip and fare totals and verifiable vehicle assignments. Operations staff require peak-hour trip volumes, cancellation rates and assignment information. Finance staff need unpaid-trip reconciliation and payment totals by method. Customer service requires complete trip records within 30 seconds to investigate disputed fares.

These requirements form the basis for the database relationships, integrity constraints and queries.

1.3 Goals and Constraints

The project defines five goals, each with an associated verification method (Table 1).

Table 1: Design goals and verification methods

Goal	Target	Verification
G1	At least 50,000 trips	Check row counts after import.
G2	Top-5 query P95 below 200 ms	Repeatedly execute the complete queries.
G3	Provisional target of 50 TPS	Define the transaction workload and conduct load tests.
G4	Identify all unpaid completed trips within 500 ms	Compare returned trip IDs with an independent expected set and measure execution time.
G5	Zero orphan records and rejection of invalid writes	Check for orphan records and test constraint violations.

These are design targets. The Top-5 queries are Q6, Q1, Q5, Q2 and Q7. Section 5 reports the available evidence, and Section 6 reviews their achievement. G2 requires repeated query measurements, while G3 requires load testing before throughput can be confirmed.

At sample scale, a single instance simplifies deployment and consistency management. The design prioritises consistent dispatch and payment records, with writes suspended during an instance outage. Failed multi-step transactions should roll back all their updates. Reporting may tolerate minute-level staleness as a design assumption, while customer-service queries require current data. Reports and writes share the same instance, so a future read replica could reduce resource contention. Availability and replica performance require further testing. Other constraints include MySQL 8.0.45 compatibility, rebuilding within 30 minutes using fixed-seed scripts, and 180-day online retention followed by archiving.

1.4 Dataset Selection

The project uses synthetic data because the public datasets considered do not provide all the identifiers, vehicle assignments and payment relationships required by the schema [2]. Fixed-seed generation allows members to rebuild the same dataset and reproduce query results.

Generation parameters are based on full-dataset RideAustin statistics for service tiers, fares and ratings [6]. The comparison is therefore descriptive. Training-only calibration lacks independently verifiable evidence. Cancellations, payment methods, unpaid rates and pickup zones are course assumptions.

The final sample contains 60,000 trips spanning one year, supporting monthly reporting demonstrations. It intentionally includes 1,442 completed trips without payment records for reconciliation tests. The sample includes historical records, while the 180-day online retention and archival policy remains a design requirement.

2 Entity-Relationship Model

2.1 ER Diagram and Notation

Figure 1 (reproduced full-page in landscape in Appendix C) shows the eight tables in our model. The five tables supplied in the Topic 5 handout remain unchanged. We added `trip_detail`, `service_tier` and `pickup_zone` for the two group-approved extension questions. The diagram follows the handout's notation: primary keys are italic, foreign keys have an asterisk, and each link has a cardinality label [1]. Blue boxes show the five base tables; green boxes show the extensions.

2.2 The Five Base Tables

Each `Trip` has one `Passenger` and one `Driver`, identified by the required `psg_id` and `driver_id` foreign keys. A passenger or driver can appear in many trips. Under decision D-05, a Trip is recorded only after a driver accepts a request, so `start_time` records acceptance rather than the passenger's request or boarding time.

Each `Driver` references one `Vehicle`. The `UNIQUE` constraint on `Driver.vehicle_id` also means that a vehicle can have at most one driver in this database. The handout's sample diagram shows a broader N:1 Driver–Vehicle relationship; for the implemented model, we follow the supplied DDL in Listing 17. Each `Payment` belongs to one Trip, while `Payment.trip_id` `UNIQUE` allows a Trip to have either no payment or one payment. The no-payment case is needed for the handout's query about completed but unpaid trips.

2.3 The Three Extension Tables

`trip_detail` connects a Trip to one `service_tier` and one `pickup_zone`; either tier or zone can be used by many detail rows. Its `trip_id` is both a primary key and a foreign key, so the database permits *at most one* detail row per Trip. The generator supplies one for every Trip, including those cancelled after acceptance. The four service-tier names come from RideAustin; `fare_multiplier` is descriptive and is not used to calculate `Trip.fare`. The 15 pickup zones are a course-defined setting, not a RideAustin field. These extensions leave all five base tables and their columns unchanged.

The pickup zone supports q09, which counts trips that reached driver acceptance, including later cancellations. It cannot count requests that never found a driver. The tier and nullable per-trip `rating` support q10, which compares completed-trip counts, average `Trip.fare`, numbers of recorded ratings and average ratings across tiers. This rating is given by a passenger for a particular trip, not a driver's historical average. The `CHECK` constraint limits recorded ratings to 1–5; the generator and data checks handle the separate rule that only completed trips can have a rating.

2.4 Normalization

We reviewed all eight tables using the 1NF–3NF definitions taught in Lecture 08. Each column holds one value (1NF). All declared candidate keys are single columns, so there is no partial dependency on a composite key (2NF). We also checked that a non-key attribute does not depend on another non-key attribute within the same table (3NF); the table-by-table review is in `01_docs/b5_normalization_review.md`.

For example, adding `psg_name` to `Trip` would give `trip_id` → `psg_id` → `psg_name`. A passenger's name would then be repeated across trips and could become inconsistent when updated. In our model, `Passenger` stores the name once and `Trip` keeps only the `psg_id` foreign key. The same principle keeps tier and zone names in their own tables.

3 Implementation

The design of Section 2 is realised by three repository artefacts: the idempotent build script `02_sql/build.sql` (§3.2), the deterministic data generator `02_sql/import_data.py` (§3.3), and a clean-room acceptance procedure with a standardised screenshot package (§3.4).

3.1 Environment and Architecture

All five members work on separate local MySQL 8 instances holding the same schema, `ride_hailing` (`utf8mb4_general_ci`); the group compatibility baseline is the lowest registered server version (8.0.45), so the scripts avoid CTEs and window functions and treat pre-8.0.16 servers as silently ignoring CHECK constraints. GitHub is the single source of truth: a schema change counts only when it lands in `build.sql` ("change the table = change the script = commit"), and after every pull members rebuild the database from scratch, keeping the five environments identical by construction.

3.2 DDL Organisation and Key Constraints

`build.sql` is organised in five blocks: (0) drop/recreate the schema; (1) the official five-table DDL of Topic 5, kept unmodified in the repository ("base immutability rule"); (2) the official seed queries; (3) the extension zone; (4) a self-check block printing the server version, the table list and per-table counts, turning every rebuild into a verifiable event. Extensions never touch the five base tables: two seeded dimension tables (`service_tier`, four tiers calibrated to RideAustin shares; `pickup_zone`, 15 zones) and one 1:1 extension table `trip_detail` whose primary key is also a foreign key to `Trip`. Table 2 maps each ER cardinality to the constraint that enforces it.

Table 2: ER cardinalities and their SQL enforcement (`build.sql`).

Relationship	Constraint	Cardinality
Trip–Payment	`Payment.trip_id NOT NULL UNIQUE` + FK	1:(0..1)
Vehicle–Driver	`Driver.vehicle_id NOT NULL UNIQUE` + FK	1:(0..1)
Passenger/Driver–Trip	NOT NULL FKs	1:N mandatory
ServiceTier/PickupZone–TripDetail	NOT NULL FKs	1:N mandatory
Trip–TripDetail	`trip_id` PK-as-FK	1:(0..1)

One design rule is not enforced by the table CHECK constraints: `rating` must be non-NULL only for completed trips — a cross-table condition on `Trip.status` that a table-level CHECK cannot reference (a trigger could express it; trigger-based enforcement was considered out of course scope). The generator and the acceptance checks enforce the rule instead.

3.3 Data Generation and Import

SQL generation uses the Python standard library only (the optional `--load` direct-import path additionally requires PyMySQL). A single seeded RNG (`seed=534`) draws status, service tier and pickup zone from exact-count pools, so those proportions are exact and only the order is random; payment method, rating presence and star values are probability draws. Fares follow the group's RideAustin-calibrated formula — base + per-minute + per-mile + a fixed $1.65 surcharge; the REGULAR tier uses a two-period mile tariff (0.99/1.50 per mile, 80/20 split) — and free cancellation (no payment row), charged cancellation ($1.50) and completed-but-unpaid trips are all represented per the fare-calibre decision. Determinism was verified by generating the import file twice and comparing checksums: the 174,698-statement `data_import.sql` is byte-identical across runs, and a rebuild on a second machine reproduced all row counts. Table 3 lists the resulting row counts.

Table 3: Row counts after a clean rebuild and import (`seed=534`).

	Passenger	Vehicle	Driver	Trip	Payment	service_tier	pickup_zone	trip_detail
Rows	3,000	150	150	60,000	51,398	4	15	60,000

3.4 Verification

Acceptance was defined as a clean-room procedure: drop the schema, run `build.sql`, import `data_import.sql`, then execute the twelve-statement acceptance script `02_sql/c5_check.sql`.

It re-derives the eight table counts of Table 3, runs zero-anomaly checks (no orphaned foreign keys, no trip without an extension row, no payment for a non-existent trip), and spot-checks business queries against pre-computed expected values: the first Q1 row (trip 1, Justin Lewis, fare 8.33, cash), exactly 150 drivers with completed-unpaid trips (Q2), and the four-tier aggregate of q10. Every check passed with observed values identical to expected. Evidence is captured per the screenshot specification: 21 verification screenshots (2026-11-01) — structure and row count for each of the eight tables, the row-count summary (Fig. 12, Appendix D), and four query spot-checks — plus the ten query-result screenshots referenced in Section 4. The procedure simultaneously closes goal G1 (60,000 trips ≥ 50,000) and demonstrates goal G5 (constraint enforcement; error-code demonstrations appear with the experiments in Section 5).

4 SQL Queries and Conceptual Q&A

4.1 Business Queries

Ten queries answer the workload, all executed on the 60,000-trip sample. All ten statements are shown exactly as executed in MySQL Workbench, together with their result sets, in Appendix D; the exact SQL text, verbatim from `02_sql/queries/`, is collected in Appendix B.1.

Q1–Q2: Payment reconciliation and collections. Q1 pairs every completed trip with its payment via the UNIQUE `trip_id` join — `fare` equals `amount` row by row (47,758 rows; balance is the most common method at 45%, cash 30%, card 25%). Q2 flips the join into a `NOT EXISTS` anti-join to find drivers with uncollected fares: all 150 drivers, 1,442 trips. Cancelled trips carrying the $1.50 fee already have a Payment row, so they stay off the list. (Screenshots: Appendix D.)

Q3–Q4: Revenue by vehicle and by driver-day. Q3 aggregates listed trip fares across `Vehicle`→`Driver`→`T` (fare is the sticker price, not cash collected); the distribution is balanced, with head vehicles at 435–445 trips versus a fleet average of 400. Q4 groups completed trips by driver and day, then ranks by total fare — the top driver-day totals $158.92. Note that SUM(fare) measures listed prices rather than driver take-home pay. (Screenshots: Appendix D.)

Q5–Q6: Collections exposure and dispatch planning. Q5 finds completed-but-unpaid trips where more than 24 hours have passed since *dispatch acceptance* (`start_time`), not since completion or payment due date — the model lacks those fields, so dispatch time is a rough proxy. All 1,442 qualify because the simulated history ends before load time. Q6 groups trips by hour and keeps hours above 2,000 trips with `HAVING`, something `WHERE` cannot do (16 of 24 hours survive); the evening block 17:00–20:00 carries 20.5% of volume (12,299 of 60,000 trips) in the generated sample. (Screenshots: Appendix D.)

Q7–Q8: Trends and retention. Q7 joins recorded payment amounts to trip dispatch months (grouped by when the trip was accepted, not when payment was recorded): the result spans thirteen calendar months — the two boundary months are partial — and the twelve full months (2025-11 to 2026-09) vary between $52.4k and $57.9k with no seasonal pattern, a property of the generated sample rather than evidence about real growth. Q8 keeps passengers active on two or more distinct days (accepted-trip activity: the query has no status filter) — all 3,000 passengers qualify, and the heaviest rides 36 trips across 36 distinct days. (Screenshots: Appendix D.)

Q9–Q10: Spatial and product-line views from the extension tables. Q9 joins through `trip_detail` to `pickup_zone`: the top three zones absorb 35.0% of accepted trips — Downtown and East Austin 7,200 each (12.0% apiece) and South Congress 6,600 (11.0%) — matching the course-assumed zone concentration. This counts accepted trips only — requests without a driver are outside the data. Q10 joins to `service_tier` and uses `COUNT(rating)` for coverage (NULL-skipping, reported as a share of completed trips) and `AVG(rating)` for the mean: average fares climb from $13.40 (REGULAR) to $35.95 (LUXURY), about 2.7×, while average ratings stay in a narrow 4.75–4.86 band at roughly 90% coverage — statistics of the generated sample, not evidence that premium products monetize without hurting satisfaction. (Screenshots: Appendix D.)

4.2 Conceptual Q&A

Q1: Primary key vs. foreign key. A primary key is NOT NULL and UNIQUE and acts as the address of a row (`Passenger.psg_id`). A foreign key is a child-table column whose values must exist in the parent key (`Trip.psg_id`). MySQL rejects `Trip(psg_id=99999)` with error 1452 if the passenger does not exist, and blocks deleting a still-referenced parent under RESTRICT (or CASCADE / SET NULL if declared). In our schema `Payment.trip_id` is NOT NULL, UNIQUE and a foreign key at once, so a payment always attaches to exactly one real trip — the 1:1 implementation itself.

Q2: 1NF, 2NF and 3NF. 1NF: each cell holds one atomic value. 2NF: no non-key attribute depends on only part of a composite key — ours use single-column surrogate keys, so 2NF holds by construction. 3NF: no transitive dependencies. If we stored the passenger name directly in `Trip(trip_id, psg_id, psg_name)`, then psg_name would depend on psg_id, which depends on the key — a transitive chain. A passenger with many trips would have the name repeated across rows; renaming requires touching every one, and deleting the last trip of a passenger would lose the name entirely. Keeping names in `Passenger` and referencing by `psg_id` avoids this, which is how our schema is structured.

Q3: INNER, LEFT and RIGHT JOIN. INNER JOIN keeps only matching rows; LEFT JOIN keeps every left-table row, filling unmatched right columns with NULL; RIGHT JOIN is the mirror image, keeping every right-table row and filling unmatched left columns with NULL. LEFT JOIN is mandatory when zero-trip drivers must appear in a per-driver report — INNER JOIN would drop them silently. The status filter goes in the ON clause (in WHERE it degenerates the join back to INNER), and the count uses `COUNT(t.trip_id)` instead of `COUNT(*)` so unmatched drivers read 0, not 1.

Q4: Subquery vs. JOIN. "Who has an accepted trip in the PREMIUM tier?" (the query spans all statuses; adding a `completed` filter would restrict it to finished rides). With a JOIN:

```
SELECT DISTINCT p.psg_id, p.name
FROM Passenger p
  JOIN Trip t ON t.psg_id = p.psg_id
  JOIN trip_detail td ON td.trip_id = t.trip_id
  JOIN service_tier st ON st.tier_id = td.service_tier_id
WHERE st.name = 'PREMIUM';
```

With a subquery:

```
SELECT psg_id, name FROM Passenger
WHERE psg_id IN (SELECT t.psg_id FROM Trip t
```

```
    JOIN trip_detail td ON td.trip_id = t.trip_id
    JOIN service_tier st ON st.tier_id = td.service_tier_id
    WHERE st.name = 'PREMIUM');
```

Both return the same 489 passengers. Neither form is inherently faster: performance depends on the execution plan, indexes and selectivity, not on syntax. EXPLAINing both on our dataset shows why — MySQL rewrites the IN-subquery into a semi-join [3], and the two plans walk the same tables in the same order (`service_tier` by unique key, `trip_detail` by index, then `Trip` and `Passenger` by primary key), differing only in how duplicates are removed (`DISTINCT` via a temporary table vs. semijoin materialisation). NOT IN is unsafe when the subquery column can be NULL; `Payment.trip_id` is NOT NULL.

Q5: ACID and concurrent payments. Atomicity: a transaction's writes all happen or none do. Consistency: committed state satisfies every constraint. Isolation: concurrent transactions do not corrupt one another's reads and writes. Durability: committed writes survive crashes. Consider two concurrent transfers out of one $100 wallet, each moving $50 to a different driver — debit the wallet, credit the driver, as one transaction. Interleaved as read-then-write, both transactions read 100, both write back 50, and one debit is silently lost: an isolation failure. If instead a transfer dies after the debit but before the credit, atomicity rolls the half-finished transaction back, so no money vanishes mid-flight. The MySQL fix for the lost update is `SELECT …FOR UPDATE` (the read takes a row lock, serialising the read-modify-write; plain SELECTs under REPEATABLE READ stay non-locking), while the UNIQUE on `Payment.trip_id` additionally blocks paying one trip twice.

Q6: Indexes and B+ trees. An InnoDB table stores its rows in the clustered index — a B+ tree over the primary key whose leaves hold the full rows. A secondary index is a separate B+ tree whose leaves store the indexed columns plus the primary-key value, so a secondary lookup descends two trees; either way a point lookup is O(log n) and range scans read the linked leaves in order. The cost: each index takes storage, every write maintains it, and the optimizer rarely uses a low-selectivity index alone. We index high-selectivity columns in WHERE, JOIN and ORDER BY, respecting the leftmost prefix (status equality before start_time range in our composite index). MySQL creates an index on foreign-key columns when the child table has none; `status` and `start_time` are not foreign keys, so the composite `(status, start_time)` of Experiment 1 was created explicitly for the test query. Experiment 1: ALL → range, 60,261 → 3,259 rows, 16.6 → 0.46 ms.

Q7: GROUP BY, HAVING vs. WHERE. WHERE filters individual rows before grouping and cannot reference aggregates. GROUP BY buckets the survivors into one output row per group. HAVING then filters whole groups after aggregation and may reference aggregates — an aggregate in WHERE is a syntax error. "Months with ≥ 1,000 completed trips" shows the split: WHERE discards non-completed rows, GROUP BY buckets by month, HAVING keeps only buckets meeting the threshold. The logical order is FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Q8: VIEW vs. STORED PROCEDURE. A view is a saved SELECT that behaves like a table but stores no data; it is expanded at query time. It suits query reuse, column-level access control, and insulation from schema changes. A stored procedure is a precompiled server-side program with parameters and control flow, invoked with CALL; its body contains semicolons, hence

the DELIMITER switch (a client-console setting, not a server statement). Our project runs its queries directly and defines neither object; as illustrations, a view `v_completed_revenue` could wrap the Q4 daily-revenue SELECT for reuse, while a procedure `sp_check_unpaid(24)` could wrap the Q5 collections check with the threshold as a parameter. A view is queried; a procedure is run.

5 Evaluation

5.1 Test Environment

All experiments ran on a MacBook Air M1 (8 GB RAM) with MySQL Community Server 26.7.0, on the 60,000-trip dataset loaded via the repository's `build.sql` and `import_data.py` (seed 534). Timings are single-run values taken from Workbench's Action Output (query execution time, excluding result-set transfer). No warm-up or repetition was applied; figures should be read as indicative magnitudes rather than stable benchmarks.

5.2 Experiment 1: Index Before/After

The target query counts completed trips accepted since 2026-10-01; the experiment then creates the composite index `idx_trip_status_time` on `Trip(status, start_time)` — both statements verbatim in Appendix B.2, and the before/after screenshot pair in Appendix D (Figure 14). With no secondary index on those columns the plan is a full table scan (`type=ALL`, rows ≈60,261). After the composite index — equality column first, range column second, leftmost-prefix rule — the plan switches to a covering index range scan (`Using index`), no table lookups.

Table 4: Index before/after on the 60,000-row `Trip` table.

	`type`	`key`	est. rows	actual time
before (no index)	ALL	—	60,261	16.6 ms
after (composite)	range	`idx_trip_status_time`	3,259	0.46 ms

Wrapping the indexed column in a function (`DATE(start_time) = ...`) degrades access to a sequential index scan; rewriting as a half-open interval restores `type=range`. This single measurement shows the index eliminates the full scan; it does not constitute a measured P95 across the Top-5 workload queries.

5.3 Experiment 2: JOIN vs Subquery

One question — count completed trips without a payment row — in three formulations: `NOT EXISTS`, `NOT IN` and `LEFT JOIN …IS NULL` (statements verbatim in Appendix B.2). Each returned a single result row with the same count, 1,442, under identical plans: a full scan of `Trip` probed against the unique index on `Payment.trip_id` (`eq_ref`), with `Extra: Not exists; Using index` — the optimizer rewrote both subquery forms into the same anti-join.

NOT IN is safe only when the subquery column is NOT NULL (`Payment.trip_id` is). Query-level consistency shows the three forms agree; it does not independently verify zero missed trips against a known expected set.

Table 5: Three anti-join formulations on the same question.

Formulation	result	plan signature	actual time
`NOT EXISTS`	1,442	`Not exists; Using index`	89 ms
`NOT IN`	1,442	same as above	82 ms
`LEFT JOIN ... IS NULL`	1,442	same type, minor `filtered` diff	90 ms

5.4 Experiment 3: Concurrent Transactions

Two Workbench windows (separate connections) on the same database, row `trip_id=92` of `trip_detail`. The interleaving:

Step	Session A	Session B
1	START TRANSACTION; SELECT rating INTO @a …; – reads 3	
2		START TRANSACTION; SELECT rating INTO @b …; – reads 3
3	UPDATE SET rating = @a+1; COMMIT;	
4		UPDATE SET rating = @b-1; COMMIT;
5	SELECT …; – reads 2	

Scenario 1 (lost update) ends at 2: A's increment is overwritten even though both UPDATEs reported "1 row(s) affected". Scenario 2 then reruns the demonstration with locking, on the same row and its leftover value of 2, this time with both sessions incrementing: A opens with `SELECT …FOR UPDATE`, so B's locking read blocks until A commits (`rating = rating + 1` writes 2 → 3); released, B's locking read is a current read — it sees the committed 3, not the stale snapshot — and writes 3 → 4. Both increments survive, and nothing was reset by hand. A third, independent comparison on row `trip_id=150` (initial value 3) contrasts isolation levels: under REPEATABLE READ the transaction keeps seeing its snapshot (still 3 after another session commits 5); under READ COMMITTED each read returns the latest committed value (5, then 3).

Table 6: Concurrency scenarios and constraint interceptions.

Scenario	Observation	What it shows
Lost update	final rating 2, one write lost	the failure mode
`FOR UPDATE`	B waits, then 2 → 3 → 4	row lock fixes it
RR re-read	still 3 after external commit	snapshot read
RC re-read	sees 5, then sees 3	reads latest commit
FK to missing trip	`ERROR 1452`	referential integrity
Duplicate `trip_id` payment	`ERROR 1062`	1:1 via UNIQUE
Rating out of 1–5 range	`ERROR 3819`	CHECK constraint

The last three rows also serve as evidence for G5: the constraints themselves stop illegal writes at the schema layer.

5.5 Descriptive Distribution Check against RideAustin

The generator's parameters were calibrated from full-dataset RideAustin statistics (Section 1.4); no train/holdout split was used. Table 7 is therefore a descriptive check that the generated

sample reproduces the marginals it was calibrated against — not an independent validation on held-out data. Methods: the RideAustin column uses all 1,494,125 rides (tier shares from `requested_car_category`; mean `total_fare`; `driver_rating` non-NULL share; 17–20h share computed from `created_date` in US Central time), and the generated column uses the 60,000-trip sample (mean `Trip.fare` over the 49,200 completed trips; rating coverage over completed trips; 17–20h share over all trips by `start_time`, the dispatch-acceptance timestamp). Results and field notes: `01_docs/d5_holdout_comparison.md` in the repository.

Table 7: Generated sample vs. full-dataset RideAustin statistics.

Dimension	RideAustin	Generated
Tier mix (R/S/P/L)	95.0 / 3.7 / 0.9 / 0.3 %	95.1 / 3.7 / 0.9 / 0.3 %
Mean fare (completed)	$13.99	$14.00 (Δ $0.01)
Rating coverage	≈90.7 %	89.9 %
Evening peak 17–20h share	≈20.5 %	20.5 %

All four dimensions agree within one percentage point, and the mean fare within one cent. The check covers marginal statistics only — not the joint distribution, long tails, or business-level external validity. Status mix, payment methods and the unpaid rate follow course-defined rules with no source counterpart.

5.6 Results Summary and Goal Recheck

Together these results partially address the measurable goals of Section 1; the goal-by-goal verdict, each at its declared verification scope, is Table 8 in Section 6.

6 Conclusion and Limitations

6.1 Goal Recheck

Table 8 reviews the goals defined in Table 1 against the available evidence.

Table 8: Goal recheck against the quantified goals of Section 1.3

#	Outcome	Evidence
G1	Sample-size target achieved, with 60,000 trips.	Imported row counts and a rebuild on a second machine.
G2	P95 remains unverified. The representative count query took 16.6 ms before indexing and 0.46 ms after.	Single-run measurement in Experiment 1.
G3	The provisional 50 TPS target remains unverified.	No defined load test or throughput measurement.
G4	Each COUNT query returned a count of 1,442. Expected trip IDs and full retrieval latency remain unverified.	Experiment 2 reports 82–90 ms. The corresponding SQL and timing scope require confirmation.
G5	Invalid writes in the tests were rejected. Full-database orphan-check results require separate evidence.	Errors 1452, 1062 and 3819 in Experiment 3.

6.2 Limitations

The synthetic sample demonstrates system behaviour under the selected generation rules. Parameters are based on full-dataset RideAustin statistics, so the comparison is descriptive and independent holdout validation remains unestablished. Unmatched requests are absent, limiting demand and matching analyses. Single-run measurements on one instance do not establish P95, throughput or production-scale performance. The 150 drivers and approximately 180 LUXURY trips also limit subgroup comparisons.

6.3 Future Work

Future work should repeat the five complete queries to measure P95 and test throughput under a defined transaction workload. An independent expected set of trip IDs would support reconciliation checks. Further extensions include recording pre-dispatch requests, evaluating a read replica and archiving historical records before removing expired online data. These extensions also require correctness and performance evaluation.

References

[1] CDS 534 Teaching Team. Cds 534 database management group project requirement. Course handout, 16 pp. (group shared drive), 2026. Official DDL, queries, and grading rubric for Topic 5.

[2] New York City Taxi and Limousine Commission. Tlc trip record data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page, 2026. Accessed 2026-10-22. Distributional reference only (decision D-10); NYC.gov Terms of Use apply.

[3] Oracle Corporation. Mysql 8.0 reference manual. https://dev.mysql.com/doc/refman/8.0/en/, 2026. Accessed 2026-10-22; in-group compatibility baseline is 8.0.45.

[4] Uber Engineering. Why uber engineering switched from postgres to mysql. Uber Engineering Blog, https://eng.uber.com/postgres-to-mysql-migration/, 2016. Source 2 of the A1 business-flow memo (2026-10-22); durable-transaction record practice cited in Section 1.

[5] Uber Technologies. Am i charged for cancelling a trip? (uber help). https://help.uber.com/en/riders/article/am-i-charged-for-cancelling-a-trip-?nodeId=5f6415dc-dfdb-4d64-927a-66bb06bc4f82, 2026. Accessed 2026-10-22. Real grace period and fee vary by city; our course simplification is a flat USD 1.50 fee (decision D-06).

[6] Natalia Zuniga-Garcia, Natalia Ruiz Juri, Randy Machemehl, James Scott, and Mauricio Tec. Evaluation of ride-sourcing search frictions and driver productivity: A spatial denoising approach. *Transportation Research Part C: Emerging Technologies*, 2019. RideAustin rides dataset (Mendeley Data, doi 10.17632/wp5gkjbvb9.1): approx. 1.49M trips, 2016-06-02 – 2017-04-13, CC0 1.0; calibration source for the synthetic generator (decision D-10). Accessed 2026-10-22.

A Contribution Statement

Member (SID)	Role	Main Contributions	Load
CHEN Yinan (3162550)	A	Business modelling: process flow, stakeholder needs, quantified goals G1–G5 and constraints; dataset selection and RideAustin calibration profile; billing and cancellation rules (D-05/D-06); workload profile and consistency trade-offs; author of Section 1; co-author of the Section 6 revisions; cross-section consistency checks; simplified business-flow diagram	20%
ZHANG Yijie (3162641)	B	ER model v1.0.1 and vector diagram; data dictionary (8 tables, 26 columns); extension proposal D-11; normalization review; ER–DDL reconciliation and signed handoff; author of Section 2 with course-notation figure; Q1/Q2 reviewer	20%
CHEN Yanwu (3162744)	C	`build.sql` (official base + extension zone + self-checks); `import_data.py` with fixed seed and RideAustin calibration (60,000 trips); clean re-run evidence on a second machine (C8); schema v1.0; engineering disciplines D-07/D-08; author of Section 3 and Appendix B	20%
LING Tong (3162677)	D	Query catalogue q01–q10 with interpretations; eight concept answer cards; three experiments (index, JOIN vs subquery, concurrency) with archived screenshots; holdout comparison table; author of Sections 4–5	20%
HU Zhongyuan (3162304)	E	Project management: timeline, milestones and follow-ups; decision registry D-05–D-12; meeting minutes (kickoff, M1, M3); four quality gates (ER / SQL / numbers / format); cover, contribution table and Section 6 co-author; integration and final compilation	20%

Table 9: Team contributions

B Full Build Script and Query Listings

Data Generation Entry

The dataset is regenerated by `import_data.py` (repository path `02_sql/`): every random choice uses the fixed seed 534, distribution parameters are the RideAustin-calibrated values of decision D-10, and the script ends with built-in acceptance checks (row counts and key ratios), so any member rebuilds identical data from the same repository scripts (constraint C6). Note that `import_data.py` *writes* the SQL load file; the third line below loads it into the database. Full rebuild on a clean machine:

Listing 1: Rebuild from scratch (idempotent, re-runnable)

```
mysql -u <user> -p < 02_sql/build.sql          # schema v1.0: base + extension + self-checks
python3 02_sql/import_data.py                  # seed=534: writes 02_sql/data_import.sql + self-checks
mysql -u <user> -p < 02_sql/data_import.sql    # load the generated 60,000-trip sample
```

Listing 2: build.sql — full DDL with repeatability self-check (schema v1.0 snapshot)

```
-- ============================================================
-- build.sql -- CDS534 group project, Topic 5: full database build script
-- Project: Ride-Hailing Order and Trip Dispatch database
-- (re-runnable: DROP + CREATE restores a clean, identical schema)
--
-- Usage: MySQL Workbench: open this file and execute all (lightning button),
--        or command line: mysql -u <user> -p < build.sql
--
-- Base rule: Section 1 is the official Topic 5 DDL (course handout Section 5,
-- Listing 17; only PDF layout whitespace cleaned, semantics identical). It is
-- never modified; extension tables are appended in Section 3 only.
--
-- Version: schema v1.0 (finalised 2026-11-05; ER v1.0.1 frozen and the
-- ER-DDL reconciliation closed). Structure is unchanged from v1.0-rc; only
-- comments and the version tag were updated.
--
-- Version policy: no forced upgrades; compatibility baseline = the lowest
-- registered member version (8.0.45, see env_record.md). Collation
-- utf8mb4_general_ci works on all versions. Below 8.0, CHECK constraints
-- would be silently ignored (still runs); queries avoid window functions
-- and CTEs in favour of JOIN / subquery forms.
-- Maintainer: C (implementation lead); changes must be committed to GitHub.
--
-- NOTE (display copy): this file is the appendix rendering of 02_sql/build.sql
-- with comments translated to English (report language). DDL and SQL
-- statements are byte-identical to 02_sql/build.sql (schema v1.0).
-- ============================================================

-- 0. Dedicated project database (isolated from personal schemas; group-wide name)
DROP DATABASE IF EXISTS ride_hailing;
CREATE DATABASE ride_hailing
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;
USE ride_hailing;

-- ============================================================
-- 1. Official base DDL (Listing 17, kept verbatim -- do not modify)
--    Five entities: Passenger / Vehicle / Driver / Trip / Payment
--    Key constraints: Driver.vehicle_id NOT NULL UNIQUE -- each driver is
--    bound to exactly one vehicle, and UNIQUE additionally caps each vehicle
--    at most one driver (the 1:1 on the ER diagram is maximum pairing; the
--    1:N arrow in the course figure is the business direction, unreachable
--    in practice). Payment.trip_id NOT NULL UNIQUE -- each trip has at most
--    one payment, and unpaid trips may exist (exactly what the official Q2
--    anti-join relies on).
-- ============================================================

CREATE TABLE Passenger (
    psg_id INT PRIMARY KEY,
    name   VARCHAR(40) NOT NULL
);

CREATE TABLE Vehicle (
    vehicle_id INT PRIMARY KEY,
```

```
    plate      VARCHAR(20) NOT NULL UNIQUE
);

CREATE TABLE Driver (
    driver_id  INT PRIMARY KEY,
    name       VARCHAR(40) NOT NULL,
    vehicle_id INT NOT NULL UNIQUE,
    FOREIGN KEY (vehicle_id) REFERENCES Vehicle (vehicle_id)
);

CREATE TABLE Trip (
    trip_id    INT PRIMARY KEY,
    psg_id     INT NOT NULL,
    driver_id  INT NOT NULL,
    fare       DECIMAL(8,2),
    status     VARCHAR(20),
    start_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (psg_id)    REFERENCES Passenger (psg_id),
    FOREIGN KEY (driver_id) REFERENCES Driver (driver_id)
);

CREATE TABLE Payment (
    pay_id  INT PRIMARY KEY,
    trip_id INT NOT NULL UNIQUE,
    amount  DECIMAL(8,2) CHECK (amount >= 0),
    method  VARCHAR(20),
    FOREIGN KEY (trip_id) REFERENCES Trip (trip_id)
);

-- ============================================================
-- 2. Official three queries (Listing 18-20, base verification)
--    On a fresh empty database, empty result sets already count as
--    "runs through" (syntax, constraints and joins are valid).
-- ============================================================

-- Q1 completed trips and their payments (1:1 join)
SELECT t.trip_id, p.name AS passenger, t.fare, py.amount, py.method
FROM Trip t
JOIN Passenger p ON p.psg_id = t.psg_id
JOIN Payment py  ON py.trip_id = t.trip_id
WHERE t.status = 'completed';

-- Q2 drivers of completed trips not yet paid (NOT EXISTS anti-join)
SELECT d.name
FROM Driver d
JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
  AND NOT EXISTS (SELECT 1 FROM Payment py WHERE py.trip_id = t.trip_id);

-- Q3 cumulative income per vehicle
SELECT v.plate, SUM(t.fare) AS earned
FROM Vehicle v
JOIN Driver d ON d.vehicle_id = v.vehicle_id
JOIN Trip t   ON t.driver_id = d.driver_id
GROUP BY v.vehicle_id, v.plate;

-- ============================================================
-- 3. Extension zone (C3, schema v1.0; built 2026-10-25, finalised 11-05)
--    Shape per D-11 (both extensions adopted) + B4 data dictionary:
--    extension attributes live in separate tables keyed by trip_id; the
--    official five tables gain no columns (base rule).
--
--    ER cardinality <-> SQL constraint mapping:
```

```
118 --      ServiceTier 1-N Trip      <-> trip_detail.service_tier_id NOT NULL + FK
119 --      PickupZone   1-N Trip      <-> trip_detail.zone_id NOT NULL + FK
120 --      Trip         1-1 TripDetail <-> trip_detail.trip_id is both PK and FK
121 --                                      (PK-is-FK -> at most one extension row
122 --                                      per Trip)
123 --      Cardinalities inside the official five tables are unchanged (1:1 on
124 --      the diagram = maximum pairing; optionality is what UNIQUE encodes):
125 --      Vehicle 1:1 Driver via vehicle_id UNIQUE -> each vehicle has 0..1
126 --      drivers; Trip 1:1 Payment via trip_id UNIQUE -> each trip has 0..1
127 --      payments (the Q2 anti-join needs unpaid rows to exist).
128 --
129 --    Generator-side invariants (import_data.py / C4; not schema-enforced):
130 --      (1) every Trip has exactly one trip_detail row, cancelled rows
131 --          included: tier and zone are request attributes fixed at dispatch
132 --          (q09 counts all Trip rows per D-11 item 1);
133 --      (2) rating is non-NULL only for status='completed' rows (~90%
134 --          coverage, D-11 item 2; AVG ignores NULLs automatically). This
135 --          rule is cross-table semantics: a CHECK cannot reference another
136 --          table's column and triggers are beyond course scope, so the
137 --          generator guarantees it and the B4 data dictionary records it
138 --          (discussed in the report as a boundary of declarative constraint
139 --          expressiveness).
140 --
141 --    Dimension seeds (4 + 15 rows) ship with build.sql rather than the
142 --    import script: tier and zone names are course settings (D-11), part of
143 --    schema semantics rather than workload data; identical across machines
144 --    without Python, and visible in C5 screenshots on an empty database.
145 -- ============================================================
146
147 -- 3.1 Service tier dictionary (names keep RideAustin's native English terms;
148 --     do not translate to local ride-hailing product names)
149 --     Calibrated mix: REGULAR ~95.0% / SUV ~3.7% / PREMIUM ~0.9% /
150 --     LUXURY ~0.3% (RideAustin full-data measurement). fare_multiplier is a
151 --     descriptive column = measured relative multiple of mean total_fare;
152 --     generated fares use the four real rate cards directly (A4 section 3),
153 --     not this column.
154 CREATE TABLE service_tier (
155     tier_id          TINYINT UNSIGNED PRIMARY KEY,
156     name             VARCHAR(16)  NOT NULL UNIQUE,
157     fare_multiplier  DECIMAL(3,1) NOT NULL
158 );
159
160 INSERT INTO service_tier (tier_id, name, fare_multiplier) VALUES
161     (1, 'REGULAR', 1.0),
162     (2, 'SUV',     1.9),
163     (3, 'PREMIUM', 2.1),
164     (4, 'LUXURY',  2.7);
165
166 -- 3.2 Pickup zone dictionary (course setting: ~15 zones, top-3 ~35% share;
167 --     spatial concentration referenced from NYC TLC; RideAustin has no zone
168 --     field, hence no real calibration source -- the report declares the
169 --     zone distribution as a course setting per D-11 item 1. Names are
170 --     placeholders: rename freely, keep zone_id)
171 CREATE TABLE pickup_zone (
172     zone_id  TINYINT UNSIGNED PRIMARY KEY,
173     name     VARCHAR(32) NOT NULL UNIQUE
174 );
175
176 INSERT INTO pickup_zone (zone_id, name) VALUES
177     (1,  'Downtown'),
178     (2,  'East Austin'),
179     (3,  'South Congress'),
180     (4,  'South Lamar'),
```

```
    (5,  'West Campus'),
    (6,  'Riverside'),
    (7,  'Hyde Park'),
    (8,  'Mueller'),
    (9,  'Airport'),
    (10, 'North Loop'),
    (11, 'Crestview'),
    (12, 'Cherrywood'),
    (13, 'Barton Hills'),
    (14, 'Zilker'),
    (15, 'Tarrytown');

-- 3.3 Trip extension table (1:1: PK-is-FK, at most one row per Trip; carries
--     both D-11 extensions -- service tier x rating and pickup zone -- so
--     q09/q10 each need a single JOIN)
CREATE TABLE trip_detail (
    trip_id         INT PRIMARY KEY,              -- PK-is-FK -> 1:1 with Trip
    service_tier_id TINYINT UNSIGNED NOT NULL,    -- request attribute, fixed at
        dispatch
    zone_id         TINYINT UNSIGNED NOT NULL,    -- pickup zone (q09 hot-zone stats
        )
    rating          TINYINT UNSIGNED NULL,        -- passenger rates driver, 1-5
        stars;
                                                   -- non-NULL only for completed
                                                      rows (~90%)
    FOREIGN KEY (trip_id)         REFERENCES Trip (trip_id),
    FOREIGN KEY (service_tier_id) REFERENCES service_tier (tier_id),
    FOREIGN KEY (zone_id)         REFERENCES pickup_zone (zone_id),
    CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5)
);

-- ============================================================
-- 4. Self-check (for screenshots): version + table list + row counts
-- ============================================================
SELECT VERSION() AS mysql_version;
SHOW TABLES;
-- Row counts right after this script (before data import): base five
-- tables = 0; dimension tables already seeded = service_tier 4 /
-- pickup_zone 15; trip_detail is generated together with Trip by C4 = 0.
SELECT 'Passenger'   AS tbl, COUNT(*) AS rows_ FROM Passenger
UNION ALL SELECT 'Vehicle',      COUNT(*) FROM Vehicle
UNION ALL SELECT 'Driver',       COUNT(*) FROM Driver
UNION ALL SELECT 'Trip',         COUNT(*) FROM Trip
UNION ALL SELECT 'Payment',      COUNT(*) FROM Payment
UNION ALL SELECT 'service_tier', COUNT(*) FROM service_tier
UNION ALL SELECT 'pickup_zone',  COUNT(*) FROM pickup_zone
UNION ALL SELECT 'trip_detail',  COUNT(*) FROM trip_detail;
```

B.1 Query Listings

The ten business queries of Section 4, verbatim from `02_sql/queries/`. The paired Workbench screenshots in Appendix D show each statement as executed, together with its result set.

Listing 3: Q1: completed trips with payment.

```
SELECT t.trip_id, t.psg_id, t.driver_id, t.fare,
       p.pay_id, p.amount, p.method
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
WHERE t.status = 'completed'
ORDER BY t.start_time DESC;
```

Listing 4: Q2: completed but unpaid drivers.

```sql
SELECT DISTINCT d.driver_id, d.name
FROM Driver d
     INNER JOIN Trip t ON t.driver_id = d.driver_id
WHERE t.status = 'completed'
      AND NOT EXISTS (SELECT 1
                      FROM Payment p
                      WHERE p.trip_id = t.trip_id)
ORDER BY d.driver_id;
```

Listing 5: Q3: total fare per vehicle.

```sql
SELECT v.plate,
       ROUND(SUM(t.fare), 2) AS total_fare,
       COUNT(*) AS trip_cnt
FROM Vehicle v
     INNER JOIN Driver d ON d.vehicle_id = v.vehicle_id
     INNER JOIN Trip   t ON t.driver_id  = d.driver_id
GROUP BY v.vehicle_id, v.plate
ORDER BY total_fare DESC;
```

Listing 6: Q4: driver daily income ranking.

```sql
SELECT driver_id,
       DATE(start_time) AS income_date,
       ROUND(SUM(fare), 2) AS total_fare
FROM Trip
WHERE status = 'completed'
GROUP BY driver_id, DATE(start_time)
ORDER BY total_fare DESC
LIMIT 10;
```

Listing 7: Q5: unpaid >24h.

```sql
SELECT t.trip_id, t.driver_id, t.fare, t.start_time,
       TIMESTAMPDIFF(HOUR, t.start_time, NOW()) AS hours_unpaid
FROM Trip t
WHERE t.status = 'completed'
      AND t.start_time < NOW() - INTERVAL 24 HOUR
      AND NOT EXISTS (SELECT 1 FROM Payment p WHERE p.trip_id = t.trip_id)
ORDER BY hours_unpaid DESC;
```

Listing 8: Q6: peak-hour volume.

```sql
SELECT HOUR(start_time) AS hr,
       COUNT(*) AS trips
FROM Trip
GROUP BY HOUR(start_time)
HAVING COUNT(*) >= 2000
ORDER BY trips DESC;
```

Listing 9: Q7: monthly revenue.

```sql
SELECT DATE_FORMAT(t.start_time, '%Y-%m') AS ym,
       COUNT(*) AS paid_trips,
       ROUND(SUM(p.amount), 2) AS revenue
FROM Trip t
     INNER JOIN Payment p ON p.trip_id = t.trip_id
GROUP BY DATE_FORMAT(t.start_time, '%Y-%m')
ORDER BY ym;
```

Listing 10: Q8: repeat passengers.

```sql
SELECT p.psg_id, p.name,
       COUNT(*) AS total_trips,
       COUNT(DISTINCT DATE(t.start_time)) AS active_days
FROM Passenger p
     INNER JOIN Trip t ON t.psg_id = p.psg_id
GROUP BY p.psg_id, p.name
HAVING COUNT(DISTINCT DATE(t.start_time)) >= 2
ORDER BY active_days DESC, total_trips DESC;
```

Listing 11: Q9: accepted trips per zone.

```sql
SELECT z.zone_id, z.name,
       COUNT(*) AS accepted_trips
FROM Trip t
     JOIN trip_detail td ON td.trip_id = t.trip_id
     JOIN pickup_zone z  ON z.zone_id   = td.zone_id
GROUP BY z.zone_id, z.name
ORDER BY accepted_trips DESC
LIMIT 15;
```

Listing 12: Q10: service tier vs. rating.

```sql
SELECT st.name AS tier,
       COUNT(*) AS completed_trips,
       ROUND(AVG(t.fare), 2) AS avg_fare,
       COUNT(td.rating) AS rated_trips,
       ROUND(COUNT(td.rating) / COUNT(*) * 100, 1) AS rating_cov_pct,
       ROUND(AVG(td.rating), 2) AS avg_rating
FROM Trip t
     JOIN trip_detail td  ON td.trip_id = t.trip_id
     JOIN service_tier st ON st.tier_id = td.service_tier_id
WHERE t.status = 'completed'
GROUP BY st.tier_id, st.name
ORDER BY completed_trips DESC;
```

B.2 Experiment Statements

The statements of the two performance experiments in Section 5, verbatim from `02_sql/exp_index.sql` and `02_sql/exp_concurrency.sql`.

Listing 13: Experiment 1: index target query and composite index creation.

```sql
SELECT COUNT(*) FROM Trip
WHERE status = 'completed' AND start_time >= '2026-10-01';
CREATE INDEX idx_trip_status_time ON Trip (status, start_time);
```

Listing 14: Experiment 2: three anti-join formulations of the same count.

```sql
-- NOT EXISTS
SELECT COUNT(*) FROM Trip t WHERE t.status = 'completed'
  AND NOT EXISTS (SELECT 1 FROM Payment p WHERE p.trip_id = t.trip_id);
-- NOT IN
SELECT COUNT(*) FROM Trip t WHERE t.status = 'completed'
  AND t.trip_id NOT IN (SELECT trip_id FROM Payment);
-- LEFT JOIN ... IS NULL
SELECT COUNT(*) FROM Trip t LEFT JOIN Payment p ON p.trip_id = t.trip_id
  WHERE t.status = 'completed' AND p.pay_id IS NULL;
```

C Full-Page ER Diagram

D Execution and Result Screenshots

Screenshots at full width, as executed in MySQL Workbench on the 60,000-trip sample. First the ten business queries of Section 4, each window showing the statement above and its result grid below; then the two build-verification screenshots of Section 3 (row-count self-check and the `trip_detail` DDL) and the Experiment 1 before/after pair of Section 5.

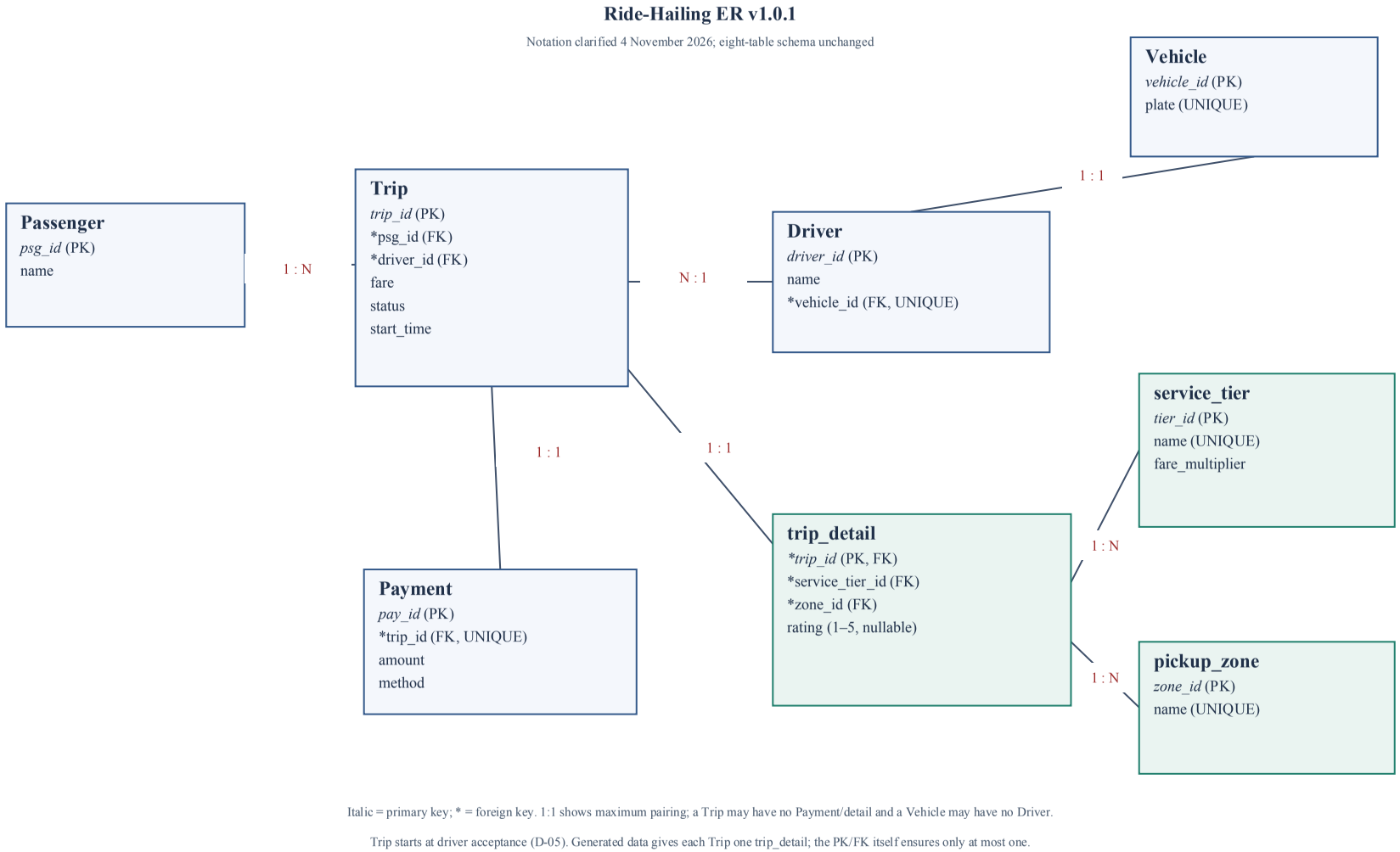


Figure 1: Final eight-table ER diagram (v1.0.1), full page. Blue boxes are the five Topic 5 tables; green boxes are the three extensions. A 1:1 label shows the maximum pairing, while the text explains when a record may be absent.

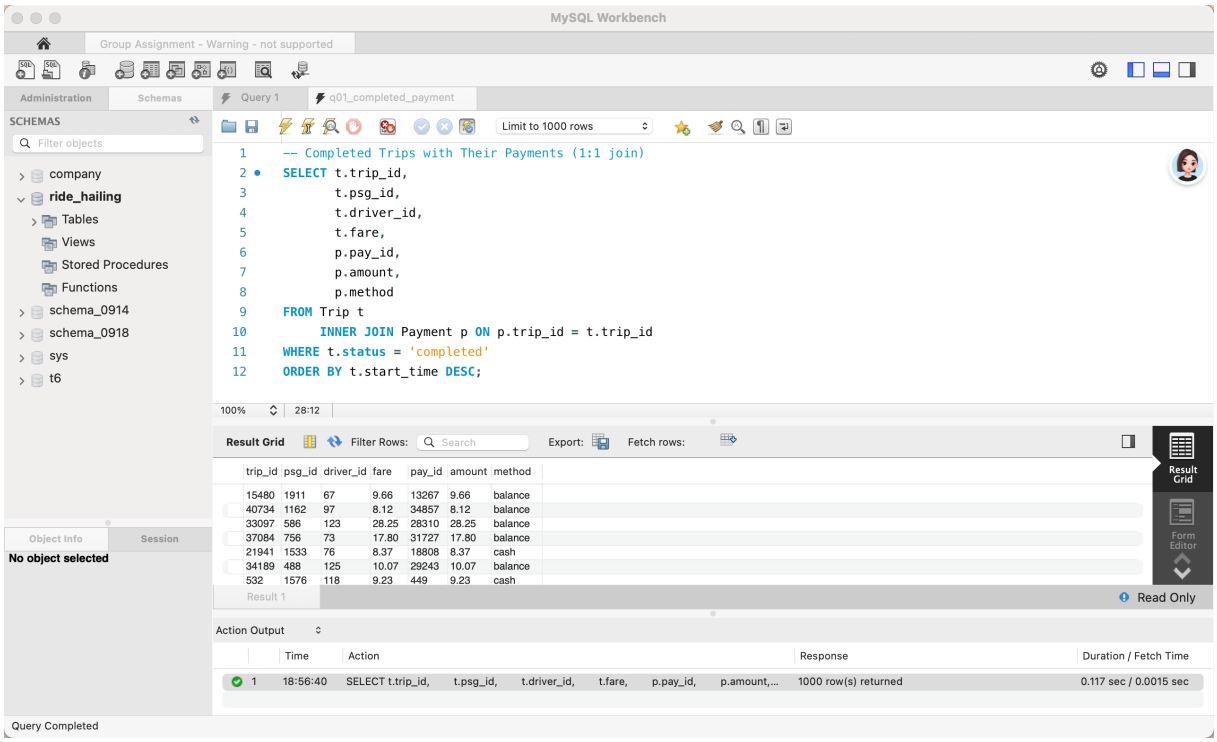


Figure 2: Q1: completed trips with payment (first 1,000 of 47,758 rows).

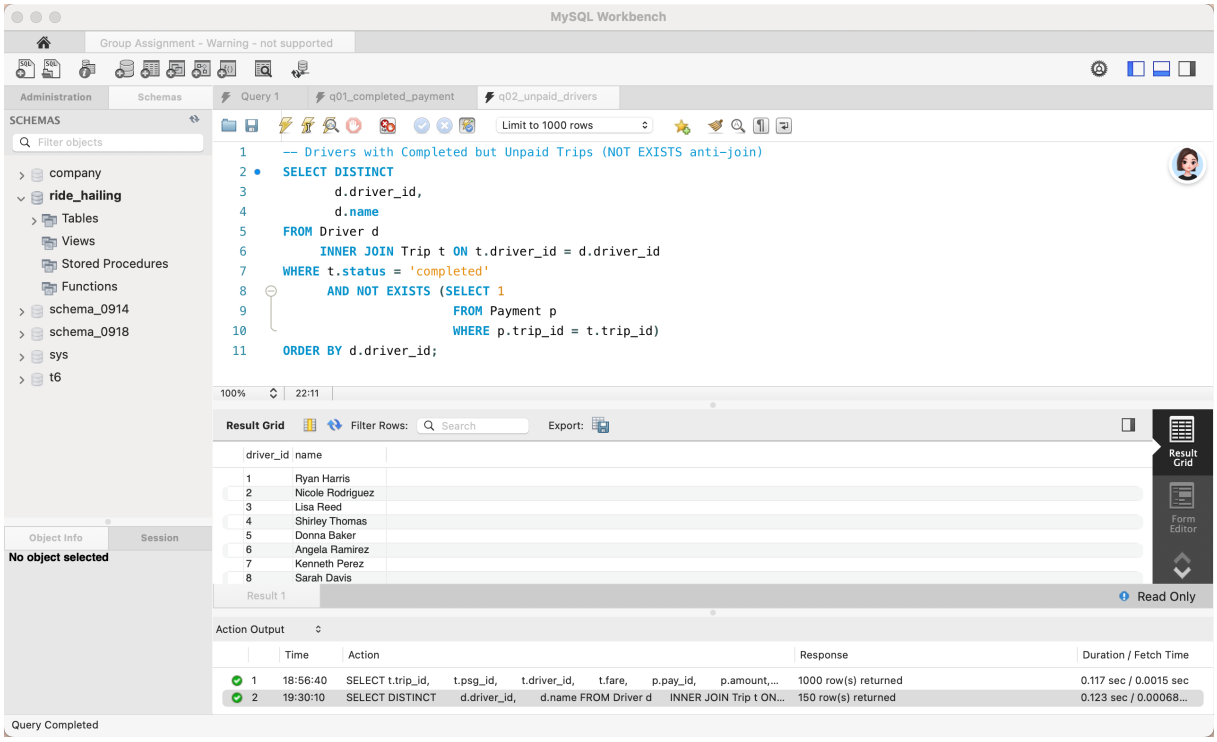


Figure 3: Q2: completed but unpaid drivers — 150 drivers, 1,442 unpaid trips.

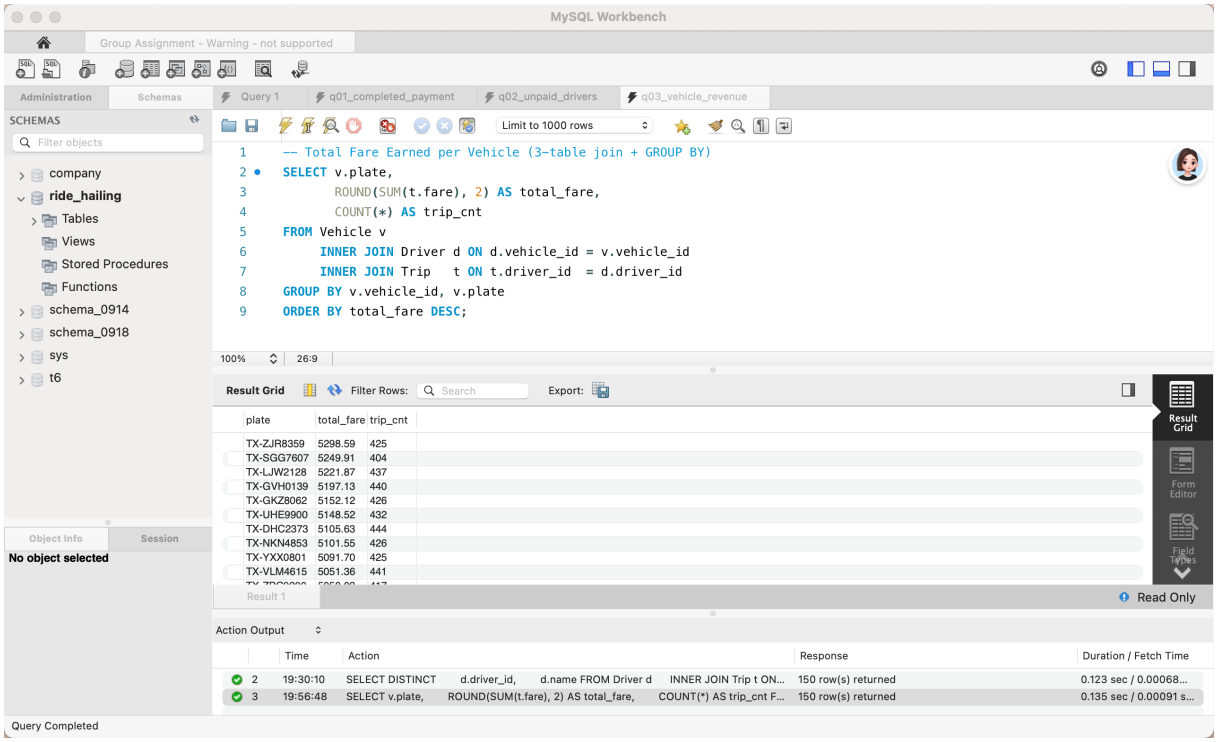


Figure 4: Q3: total fare per vehicle, 150-vehicle fleet.

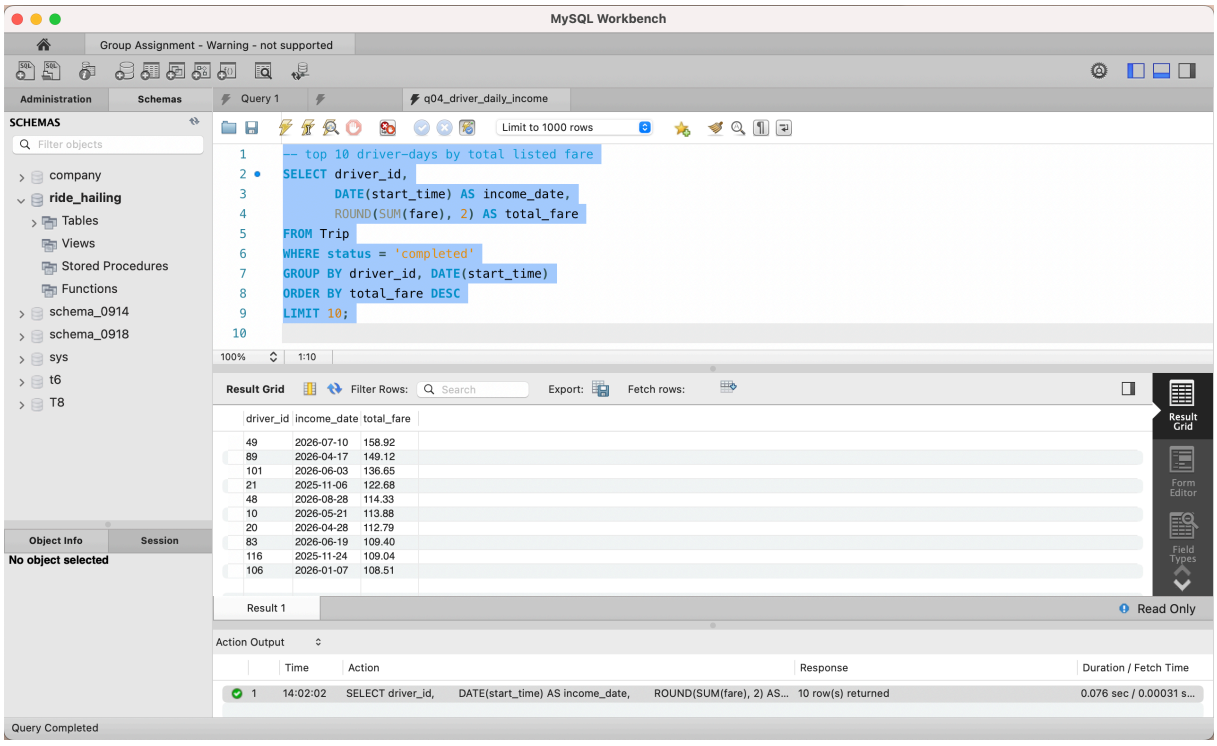


Figure 5: Q4: driver daily income ranking (GROUP BY formulation), top-10 driver-days.

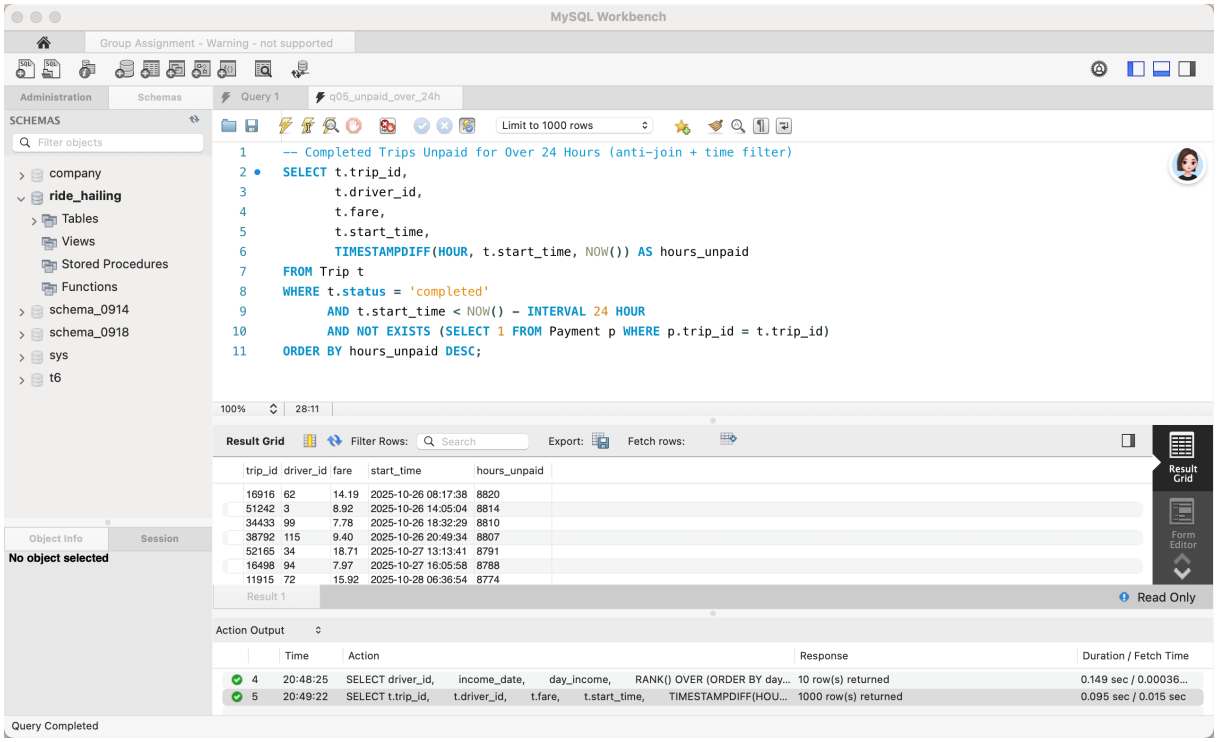


Figure 6: Q5: completed and unpaid beyond 24 hours since dispatch acceptance (first 1,000 of 1,442 rows).

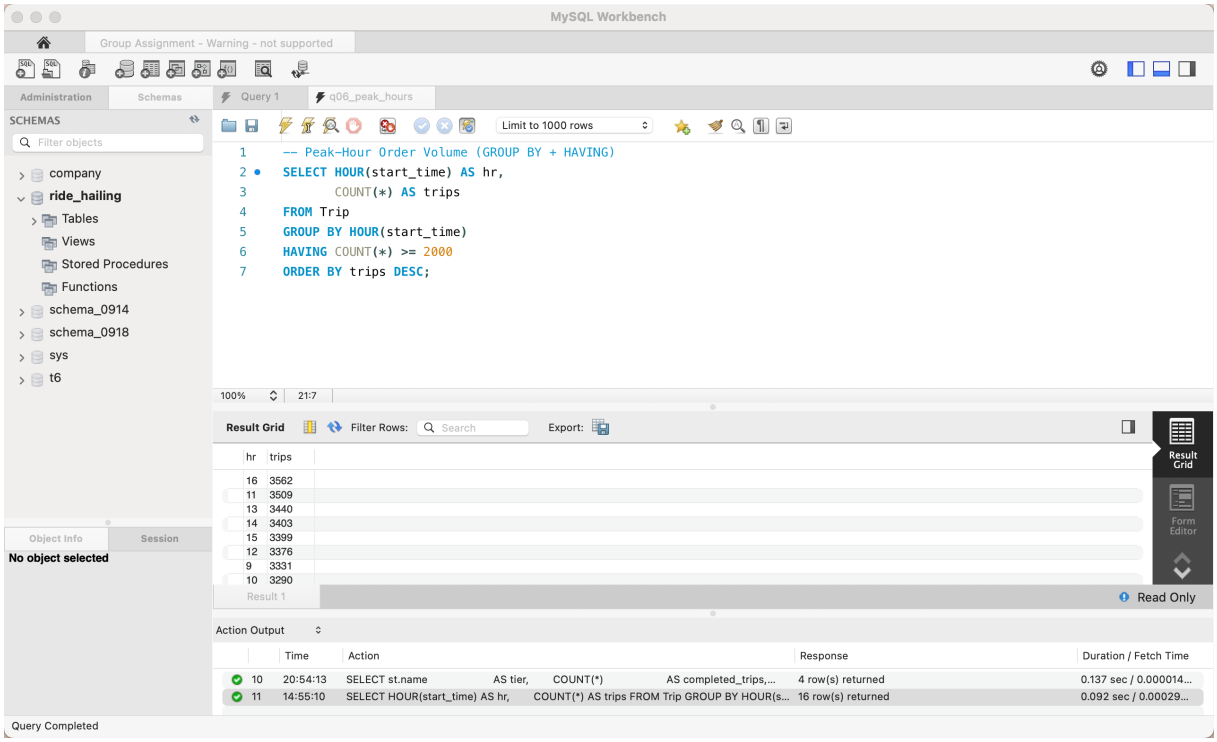


Figure 7: Q6: peak-hour volume, 16 hours above the 2,000-trip threshold.

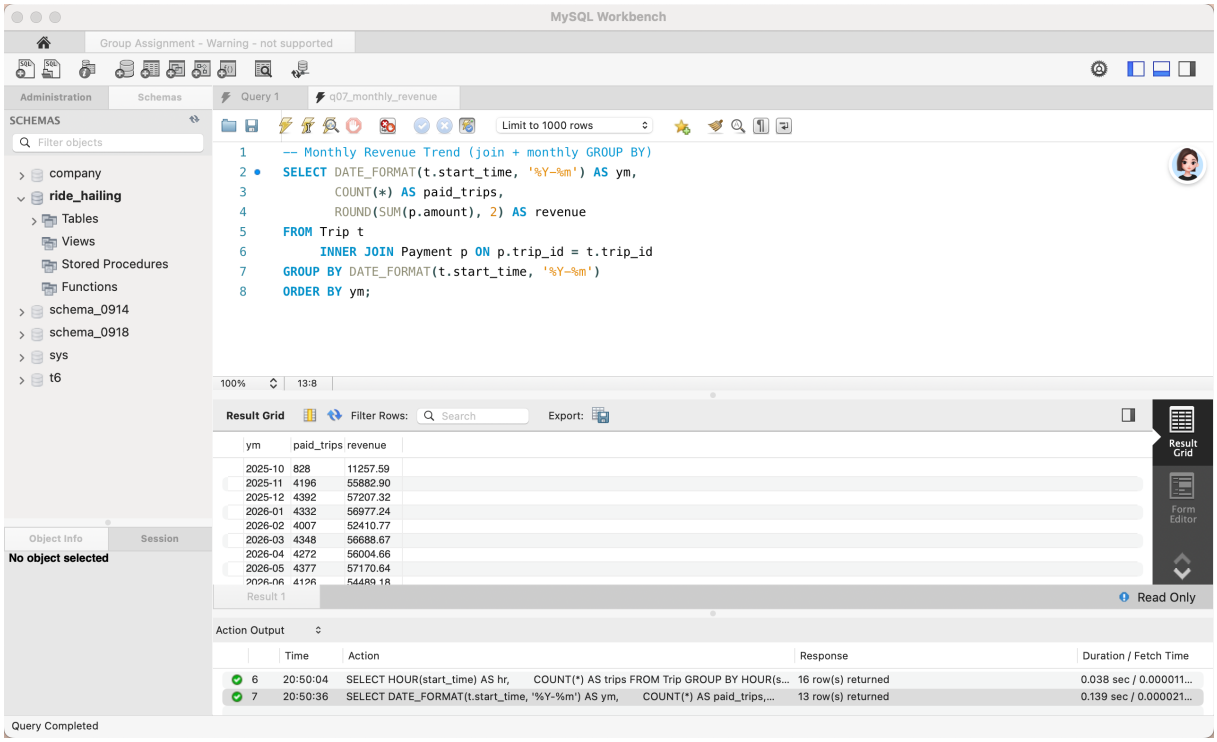


Figure 8: Q7: recorded payment amounts by trip dispatch month, thirteen calendar months (two partial).

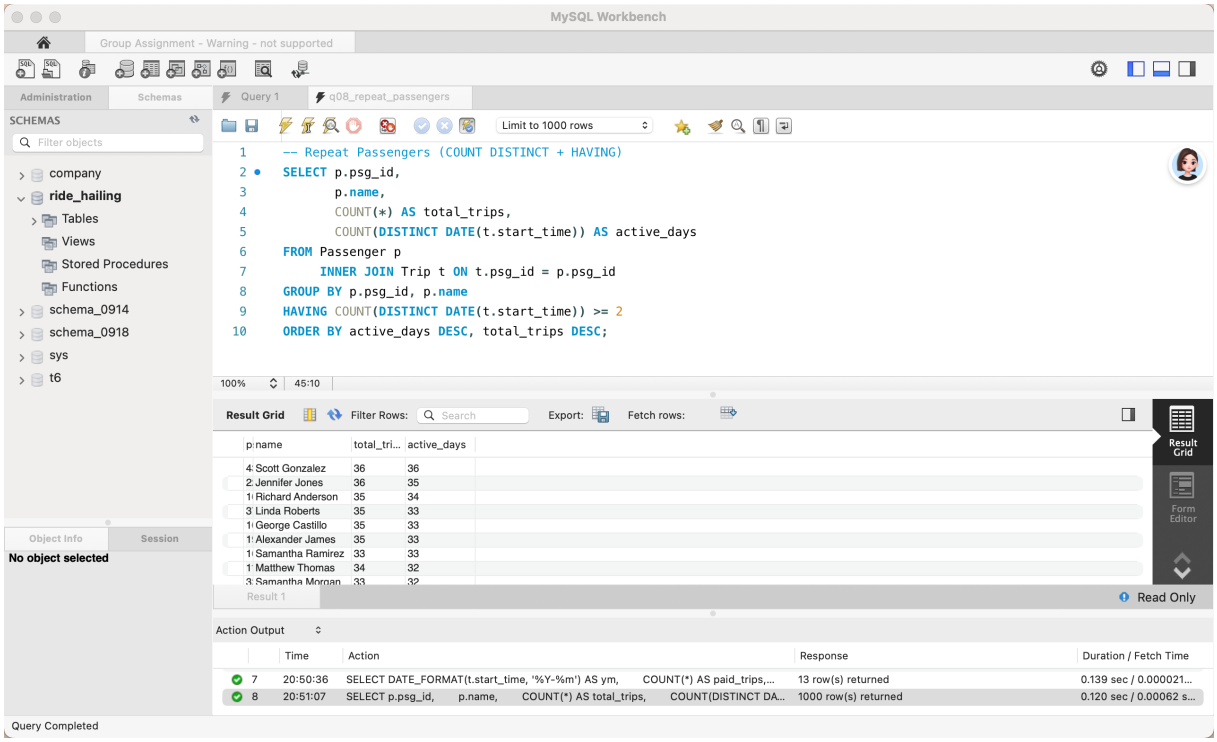


Figure 9: Q8: repeat passengers (first 1,000 rows).

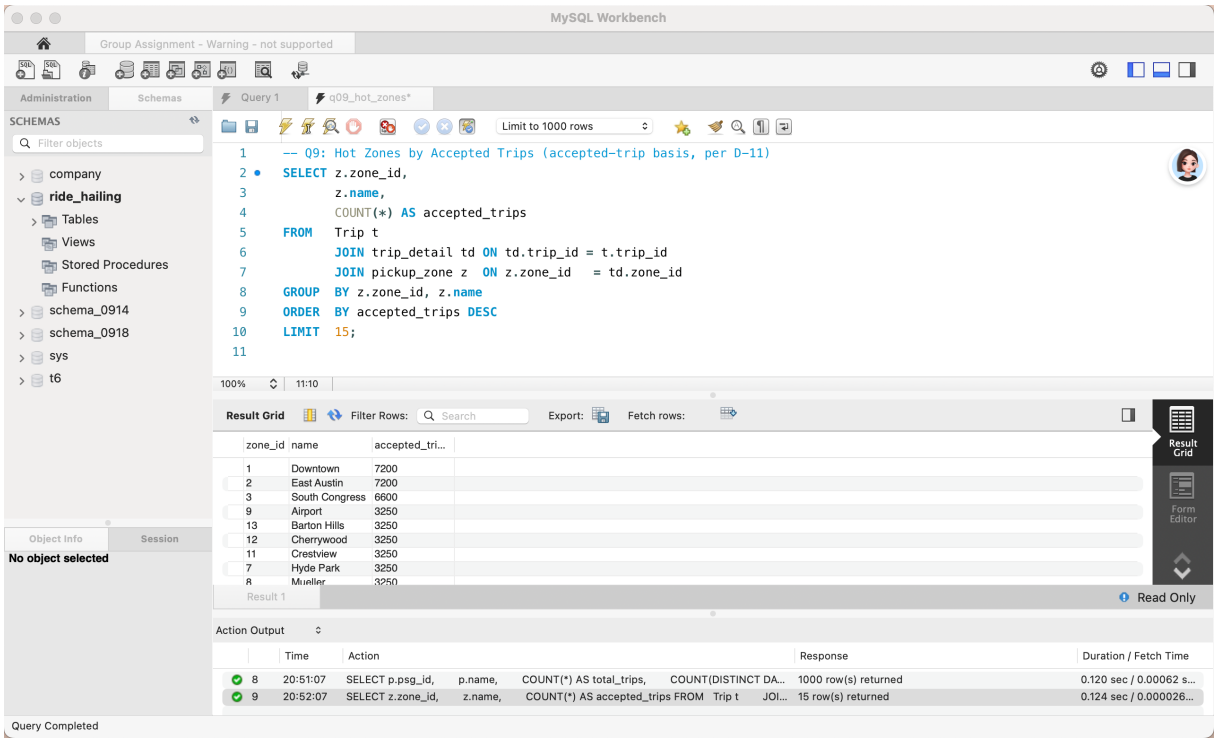


Figure 10: Q9: accepted trips per pickup zone, 15 zones.

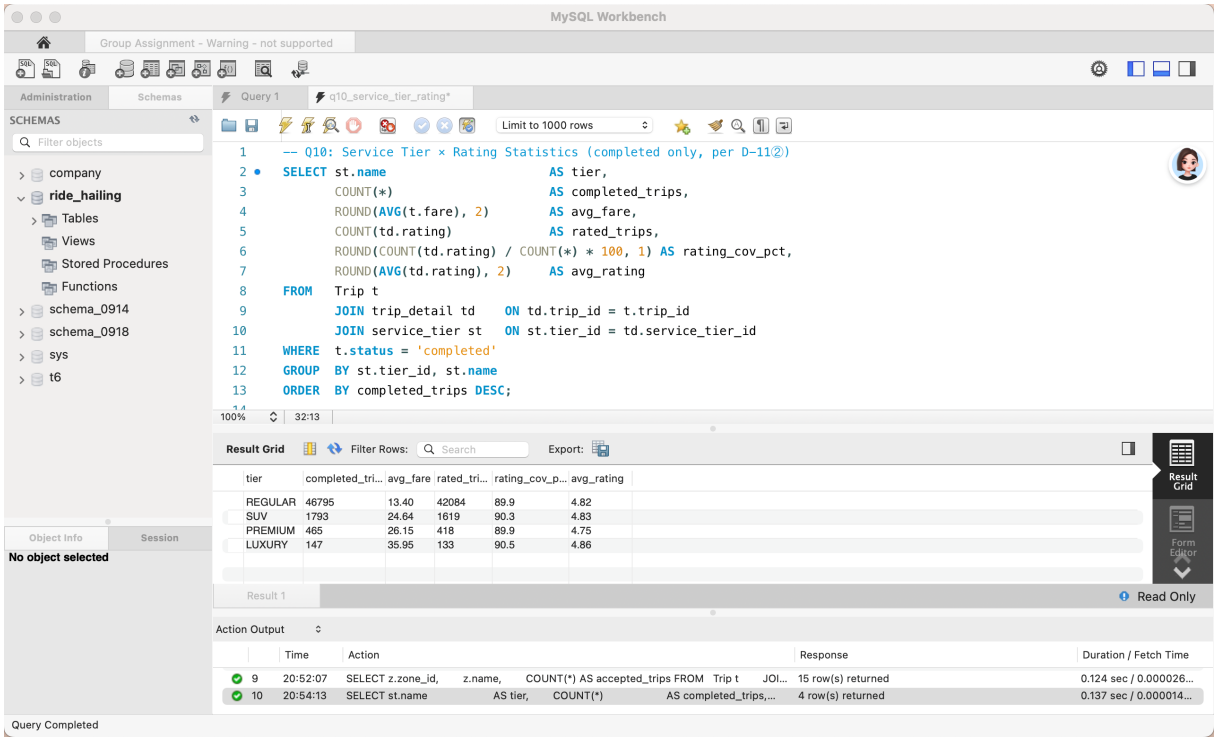


Figure 11: Q10: service tier versus rating, four tiers.

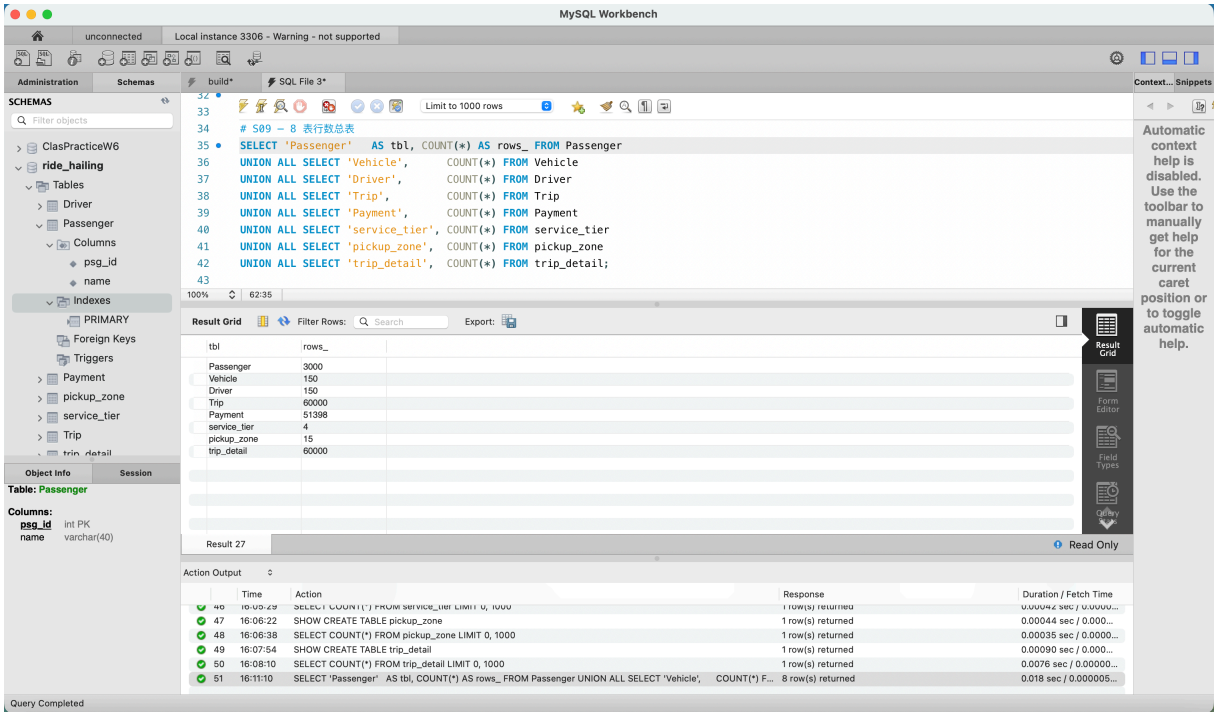


Figure 12: Eight-table row-count self-check (Section 3 acceptance run).

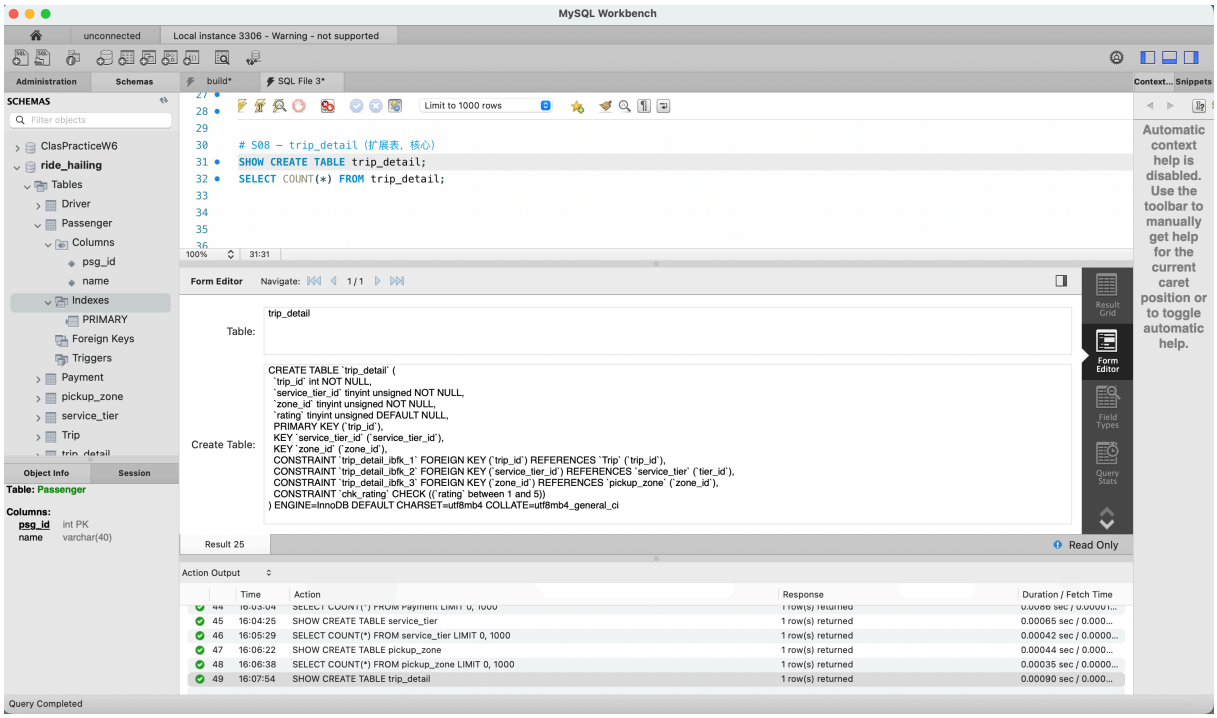


Figure 13: `trip_detail` DDL: PK-as-FK, three FKs, rating CHECK.

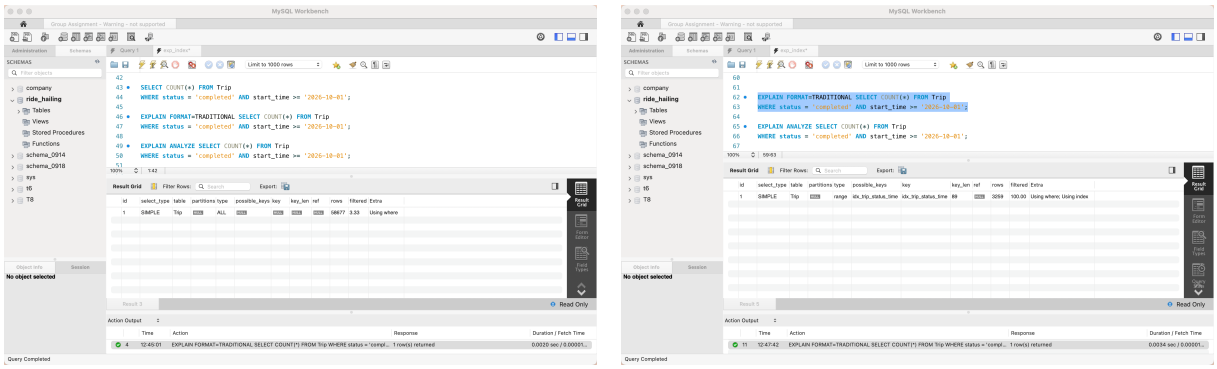


Figure 14: Experiment 1 before/after (Section 5): `type` ALL → range, `rows` 60,261 → 3,259.

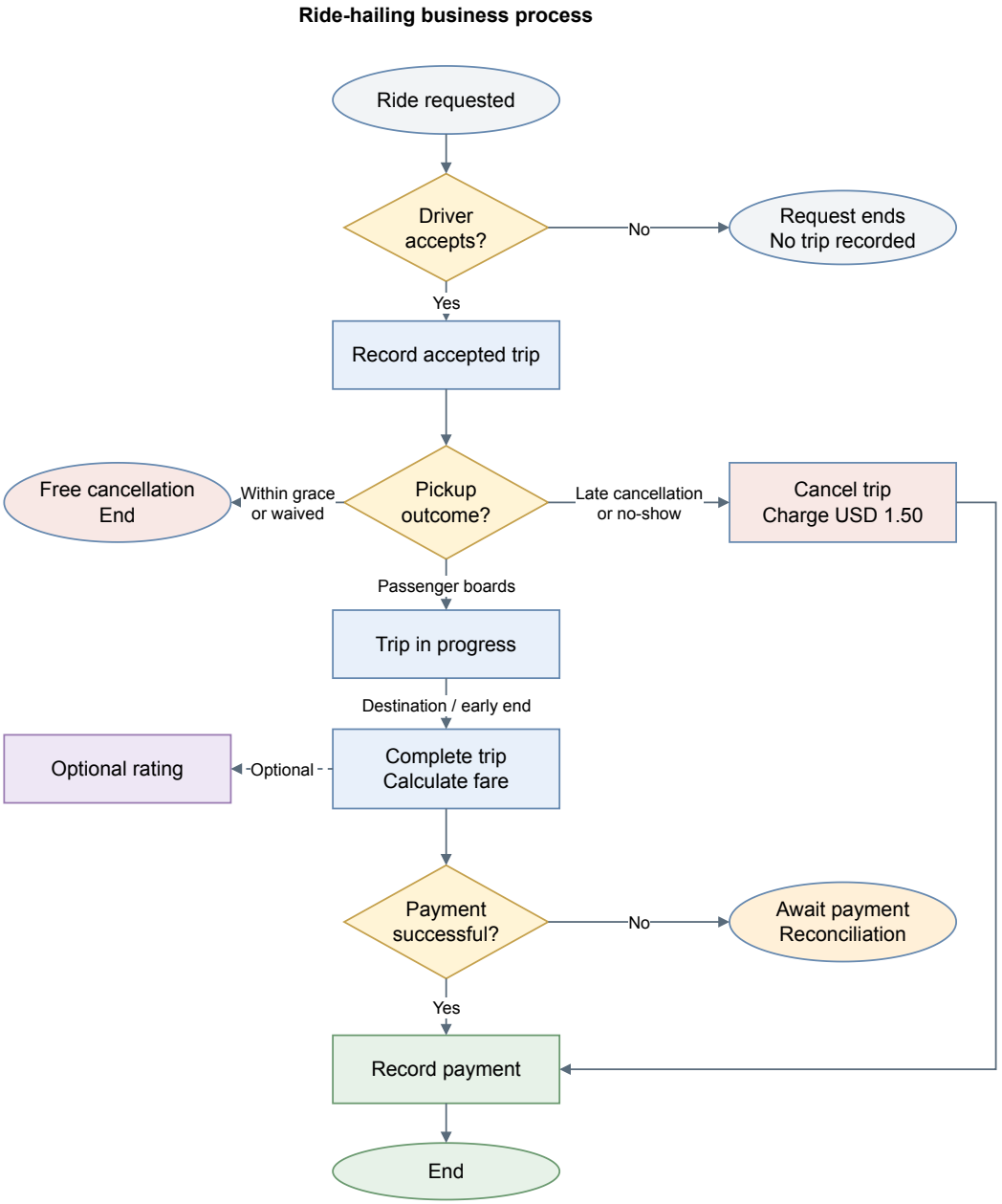


Figure 15: Ride-hailing business process. Unmatched requests remain outside the database. Dashed flow indicates an optional rating, independent of payment.